The Journal of Finance and Data Science 11 (2025) 100175

Contents lists available at ScienceDirect

The Journal of Finance and Data Science

journal homepage: www.keaipublishing.com/en/journals/jfds

Research Article

Finding money launderers using heterogeneous graph neural networks

Fredrik Johannessen [a], Martin Jullum [b,*]

[a] DNB, P.O. Box 1600, Sentrum, NO-0021, Oslo, Norway
[b] Norwegian Computing Center, P.O. Box 114, Blindern, NO-0314, Oslo, Norway

ARTICLE INFO

Keywords:
Graph neural networks
Anti-Money laundering
Supervised learning
Heterogeneous graphs
PyTorch geometric

ABSTRACT

The finance industry depends on effective anti-money laundering (AML) systems to ensure compliance and maintain operational efficiency. However, existing AML systems, which are predominantly rule-based, frequently struggle to detect money laundering accurately.

In particular, their inability to learn from historical data and properly account for diverse customer behavior is problematic. Also accounting for the vast amounts of transactional data generated daily, this challenge calls for big data analytics and advanced machine learning techniques.

In line with this, the present paper explores a graph neural network (GNN) approach, a state-of-the-art machine learning technique, to identify money laundering activities within a large heterogeneous network constructed from real-world bank transactions and business role data from DNB, Norway's largest bank.

To this end, we extend the (homogeneous) Message Passing Neural Network (MPNN) architecture to operate on a heterogeneous graph, and demonstrate its strong performance in detecting money laundering activities.

We showcase the suitability of utilizing GNN methodology to improve electronic surveillance systems for detecting money laundering, thereby contributing a pioneering approach to AML through the application of advanced data science techniques.

To the best of our knowledge, this is the first publication applying heterogeneous GNNs for AML purposes with a large real-world heterogeneous network.

1. Introduction

Money laundering is the activity of securing proceeds of a criminal act by concealing where the proceeds come from. The ultimate goal is to make it look like the proceeds originated from legitimate sources. Money laundering is a vast global problem, as it enables all types of crime where the goal is to make a profit. Because most money laundering goes undetected, it is difficult to quantify its effect on the global economy. However, a research report by United Nations – Office on Drugs and Crime (2011) estimates that 1–2 trillion US dollars are being laundered each year, which corresponds to 2–5% of global gross domestic product. Both national and international anti-money laundering (AML) laws regulate electronic surveillance and reporting of suspicious transaction activities in financial institutions. The purpose of the surveillance is to detect *suspicious activities* with a high probability of being related to money laundering,

* Corresponding author.
E-mail addresses: fredjo89@gmail.com (F. Johannessen), jullum@nr.no (M. Jullum).
Peer review under the responsibility of KeAi Communications Co., Ltd.

https://doi.org/10.1016/j.jfds.2025.100175
Received 6 December 2024; Received in revised form 27 November 2025; Accepted 16 December 2025
Available online 23 December 2025

such that they can be manually investigated. The manual investigation is performed by experienced investigators who inspect several aspects of the case, often involving multiple implicated customers, and then decide whether the behavior warrants reporting to the authorities. See Section 2 for more details about this process. The present paper concerns the electronic surveillance process, which ought to identify a relatively small number of suspicious activities in a vast ocean of legitimate transactions.

In the past few decades, the electronic surveillance systems in banks have primarily consisted of a set of rules, created by domain experts, that use fixed thresholds and a moderate number of if/else statements that determine whether an alert is generated. Typical areas the rules focus on are transactions exceeding a certain amount, frequent transfers to high-risk countries, and rapid movement of funds between accounts. Here is an illustrative example of such a rule:

> If
> – the transaction is above $10,000, and
> – it is made to a country with increased AML-risk, and
> – the customer making the transaction is classified as a retail customer,
> then trigger an alert.

Rules and thresholds are periodically reviewed and adjusted based on new trends, typologies, and regulatory changes. Such rule-based systems are still in large parts what makes up the surveillance systems (Chen et al., 2018b).

However, these rules fall short of providing efficiency (Fruth, 2018), for at least three key reasons: a) They rely on manual work to create and keep the rules up to date with the dynamically changing data (Chen et al., 2018b). This work increases with the complexity and number of rules. b) They are typically too simple to be able to detect money laundering with high precision, resulting in many low-quality alerts, i.e. *false positives*.[1] c) Their simplicity makes them easy to circumvent for sophisticated money launderers, resulting in the possibility that severe crimes go undetected. The consequence of these three shortcomings is that banks spend huge resources on an inefficient approach, and only a tiny fraction of illegal proceeds are being recovered (Pol, 2020).

Driven by multiple money laundering scandals in recent years,[2] the shortcomings of the rule-based system are high on the agenda for regulators and financial institutions alike, and considerable resources are devoted to developing more effective electronic surveillance systems. One avenue that is explored is to use machine learning (ML) to automatically learn when to generate alerts (see e.g. Jullum et al., 2020; de Jesús Rocha-Salazar et al. (2021)). ML excels at detecting complex patterns in vast volumes of data. As a consequence, ML-based systems have the potential to provide detection systems with increased accuracy and the ability to identify more sophisticated ways of laundering money.

Money laundering often involves multiple parties collaborating to conceal the origin of criminally obtained proceeds through complex chains of bank transactions across different accounts and entities (Bolton and Hand, 2002; Deprez et al., 2025, Sec 4.). These patterns are difficult to detect when entities are viewed in isolation, but become apparent when relationships between the entities are leveraged. Reflecting this, there has lately been an increased focus on network science as a tool to detect money laundering, and studies indicate that this approach has high potential (Colladon and Remondi, 2017; Li et al., 2017; Savage et al., 2017). In networks relevant to AML, the nodes can represent individuals, organizations, and addresses, while the edges can represent for instance money transfers, business ties, and joint ownership.

The simplest way to incorporate network characteristics into ML methods is to create node features that capture information about the node's role in the network, e.g. network centrality metrics or characteristics of its neighborhood. The features can then be used in a downstream machine learning task. This approach is, however, suboptimal as the generated features might not be the ones most informative for the subsequent classification, and the full richness of the relational data will in any case not be passed over to the entity-based machine learning task.

Graph Neural Networks (GNN) is a class of methods that overcome this drawback by applying the machine learning task directly on the network data through a neural network. GNNs are able to solve various graph-related tasks such as node classification (Kipf and Welling, 2016a), link prediction (Zhang and Chen, 2018), graph clustering (Wang et al., 2017), graph similarity (Bai et al., 2019) as well as unsupervised embedding (Kipf and Welling, 2016b). The primary idea behind GNNs is to extend the modern and successful techniques of *artificial neural networks* (ANNs) from regular tabular data to that of networks. In addition to their ability to incorporate both entity features and network features into a single, simultaneously trained model, most GNNs scale linearly with the number of edges in the network, making them applicable to large, sparse networks (Wu et al., 2020). Another desirable property of GNNs is that they are inductive rather than transductive: While transductive models (e.g. Node2Vec (Grover and Leskovec, 2016)) can only be used on the specific data that was present during training, inductive models can be applied to entirely new data without having to be retrained. This is crucial for the AML application where new transactions (edges) appear continuously, and customers (nodes) enter and leave on a daily basis. Given these advantages, GNNs are widely considered state-of-the-art for machine learning on graphs,

[1] Hiring more workers to inspect each alert manually is often the (unsatisfactory) resolution to compensate for this shortcoming.

[2] As an example, it was revealed in 2018 that the Estonian branch of Danske Bank, Denmark's largest bank, carried out suspicious transactions for over €200 billion during 2007–2015 (Bjerregaard and Kirchmaier, 2019). Subsequent lawsuit claims have amounted to over €2 billion.

particularly for node classification where node features are available (Chami et al., 2022), and make them well-suited for AML applications.

Most GNNs are developed for *homogeneous* networks with a single type of entity (node) and a single type of relation (edge). In the present AML use case, the relevant network is *heterogeneous* in both nodes and edges: The nodes represent both private customers, companies, and external accounts, while the edges represent both financial transactions and professional business ties between the nodes. There exist some GNNs in the literature that are able to handle heterogeneous networks, such as: RGCN (Schlichtkrull et al., 2018), HAN (Wang et al., 2019), MAGNN (Fu et al., 2020), HGT (Hu et al., 2020), and HetGNN (Zhang et al., 2019). A common issue with all these methods is that they are not designed to incorporate edge features. In our AML use case, this corresponds to properties of the financial transactions (or the business ties) and is crucial information for an effective learning task. Thus, based on our current knowledge and research, there exists no directly applicable GNN method for our AML use case.

1.1. The present paper

The present paper considers an AML scenario from Norway's largest bank, DNB, where the purpose is to detect money laundering activities. The data consists of a large heterogeneous network created from real-world bank transactions and business role data. The nodes represent bank customers and transaction counterparties, while the edges represent bank transactions and business ties. The network has a total of 5 million nodes and 10 million edges. Among the bank customers, some are known to conduct suspicious behavior related to money laundering. While the rest are assumed not fraudulent, there is certainly undetected suspicious behavior among them. Thus, this is actually a semi-labeled dataset.

There are primarily two categories of bank customers: individual (retail) customers and organization (corporate) customers. Due to the fundamental differences between these two groups and their fraudulent behaviors, it is common practice to study and model their fraudulent behavior separately. In our setting, we, therefore, treat them as two distinct node types, each with its unique set of features. In this paper, we limit the scope to modeling, predicting, and detection of fraudulent *individual* customers. I.e. all fraudulent customers in our dataset are individual customers. We chose to restrict our analysis to individuals because there is a significantly larger number of known fraudulent individuals compared to organizations, providing the model with more fraudulent behavior to learn from. Furthermore, individuals are a more homogeneous group with simpler customer relationships compared to organizations, making them a more suitable group to apply our methodology to. Nevertheless, note that the organizations in our network may very well be involved and utilized by fraudulent individuals, and possess key connections in the graph that the model learns from. In addition to network data, the nodes and edges have sets of features that depend on what type of node and edge they are.

To detect suspicious customer behavior in this dataset, we develop a heterogeneous GNN architecture. We denote our proposed GNN method *Heterogeneous Message Passing Neural Network* (HMPNN) as it expands the (homogeneous) MPNN method (Gilmer et al., 2017) to a heterogeneous setting. The extension essentially connects, and simultaneously trains multiple MPNN architectures, each working for different combinations of node and edge types. We consider two distinct approaches to aggregate the embeddings of node-edge combinations in the final step: 1) A simple summation of the embedding vectors derived from the various combinations, and 2) A concatenation of embeddings before applying an additional single-layer neural network to enhance the aggregation process. In this paper, HMPNN is compared to other state-of-the-art GNN methods and achieves superior results for our data.

The main contribution of the present paper is the application of the HMPNN method on our real-world AML use case, demonstrating the general suitability of utilizing heterogeneous GNN methodology for detecting money laundering activities – a critical challenge in today's financial sector. To the best of our knowledge, this is the first published work to utilize graph neural networks on such a large, real-world heterogeneous network for AML purposes.

The rest of this article is organized as follows: Section 2 provides some background and an overview of related work within the AML domain, as well as the GNN domain. In Section 3 we formulate our HMPNN model after introducing the necessary mathematical framework and notation. Section 4 presents the data for our AML use case, the setup of our experiment, and presents and discusses the results we obtain. Finally, Section 5 summarizes our contribution, and provides some concluding remarks and directions for further work. Additional model details are provided in Appendix A - D.

2. Background and related work

In this section, we provide some background on the AML process and give an overview of earlier, related work in the domain of AML. We also briefly describe the state-of-the-art of homogeneous and heterogeneous GNNs in the general case.

2.1. AML process

As mentioned in the introduction, financial institutions are required by law to have effective AML systems in place. Fig. 1 illustrates a typical AML process in a bank. The electronic surveillance system generates what is called *alerts* (1) on some of the transactions, or transaction patterns, that go through the bank. An alert is first subject to a brief initial inspection, where those that can easily be identified as legitimate are picked out and marked as closed. Otherwise, the alert is upgraded to a *case* (2). At this stage, multiple alerts might be merged into a single case, which regularly involves multiple implicated customers. The case is then inspected thoroughly by experienced investigators. If money laundering is ruled out, the case will be marked as closed. Otherwise, the case will be upgraded to a *report* (3), and the revealed circumstances are reported in detail to the national *Financial Intelligence Unit* (FIU) (Bartolozzi et al., 2022). From here on, the FIU oversees further action and will determine whether to start a criminal investigation.

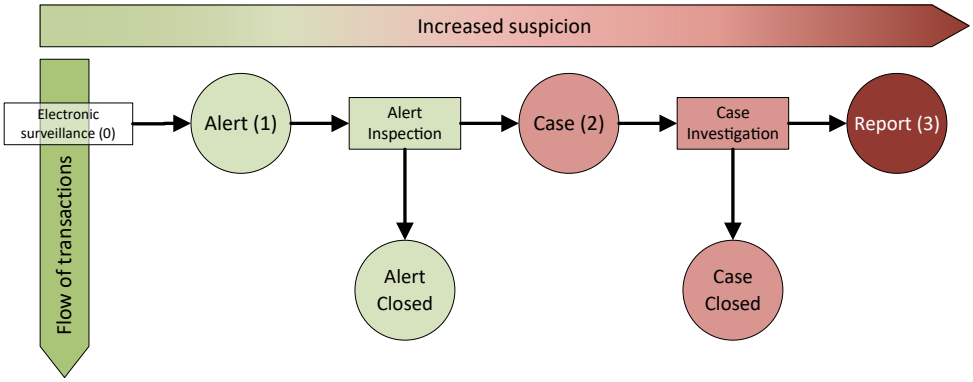

Fig. 1. Workflow following detections from the electronic surveillance system.

The manual investigation part of the process is difficult to set aside for two reasons: a) The investigation stage relies heavily on the professional judgment of experienced AML analysts, who often incorporate contextual knowledge, external information sources, and case-specific nuances that are not captured in structured data and therefore cannot be easily quantified or encoded for automated models. b) AML laws do typically not allow automated reporting of suspicious behavior. The electronic surveillance component that generates alerts is suitable for automation, and improvements in alert quality can both reduce operational costs and lower the risk that money-laundering activity goes undetected. That is also why this paper is concerned with the electronic surveillance aspect of the AML process.

2.2. AML literature

In AML literature, there are very few papers that have datasets with real money laundering cases, and the majority of articles are validated on small datasets consisting of less than 10,000 observations (Chen et al., 2018b).

Both supervised (Liu et al. (2008); Deng et al. (2009); Zhang and Trubey (2019); Jullum et al., 2020; Garin and Gisin (2023)) and unsupervised (Lorenz et al. (2020); de Jesús Rocha-Salazar et al. (2021)) machine learning have been applied to AML use cases. However, the literature is quite limited, particularly for papers utilizing relational data.

Some AML papers apply the aforementioned strategy of generating network features used in a down-stream machine learning task: Savage et al. (2017) create a graph from cash- and international transactions reported to the Australian FIU (AUSTRAC), from which they extract communities defined by filtered k-step neighborhoods around each node, to then create summarizing features used to classify community suspiciousness using supervised learning. Colladon and Remondi (2017) construct multiple graphs (each of which aims to reveal unique aspects) from customer and transaction data belonging to an Italian factoring company. On these graphs, they report significant correlation between traditional centrality metrics (e.g. betweenness centrality) and known high-risk entities. Elliott et al. (2019) uses a combination of network comparison and spectral analysis to create features that are applied in a downstream learning algorithm to classify anomalies.

There are only a few papers applying GNNs to money laundering detection: In a brief paper, Weber et al. (2018) discusses the use of GNN on the AML use case. The paper provides some initial results of the scalability of *Graph Convolutional Network* (GCN) (Kipf and Welling, 2016a) and FastGCN (Chen et al., 2018a) for a synthetic dataset. However, no results on the performance of the methods were provided. Weber et al. (2019) compare GCN to other non-relational ML methods on a real-world graph generated from bitcoin transactions, with 203k nodes and 234k edges. The authors highlight the usefulness of the graph data and find that GCN outperforms logistic regression, but it is still outperformed by random forest. The dataset, called the Elliptic dataset, is also released with the paper and has later been utilized by several others: Alarab et al. (2020) apply GCN extended with linear layers and improve significantly on the performance of the GCN model in Weber et al. (2019). Lo et al. (2023) apply a self-supervised GNN approach to create node embedding subsequently used as input in a random forest model, and reported good performance. Others (Pareja et al., 2020; Alarab and Prakoonwit, 2022) exploited the temporal aspect of the graph to increase the performance on the same dataset.

It is unknown to what extent results from synthetic or bitcoin transaction networks are transferable to the real-life application of transaction monitoring of bank transactions. The graph in this paper (which will be introduced in Section 4) is also about 25 times larger than the Elliptic dataset. Moreover, as we will review immediately below, much has happened in the field of GNN since the introduction of GCN in 2016. Finally, to the best of our knowledge, there are no papers applying GNN (or other methodology) to an AML use case with a *heterogeneous* graph.

2.3. General case GNN literature

While the history of GNNs can be traced back about twenty years, GNNs have during the past few years surged in popularity. This was kicked off by Kipf and Welling (2016a) who introduced the popular method GCN. The core dynamics in GNNs is an iterative

approach where each node receives information from its neighbors, and combines it with its own representation to create a new representation, which will be forwarded to its neighbors in the next iteration. This is called *message passing*. When node classification is the task of interest, these node representations are used to make inference about the node.

Concentrating on today's most popular group of GNNs, *Spatial-based convolutional GNNs*, we briefly review some relevant homogeneous and heterogeneous GNN methods below.

2.3.1. Homogeneous GNN literature

The GCN method (Kipf and Welling, 2016a) was originally introduced as a spectral convolutional approach for semi-supervised learning on graphs, formulated in a transductive setting and operating directly on the graph's adjacency matrix. However, the method can be reformulated in the inductive and spatial-based GNN setting using a message-passing formulation: The message received by a node from its neighbors is a weighted linear transformation of the neighboring node representations, followed by aggregating the result through summation. GraphSage (Hamilton et al., 2017) expands on GCN in two ways: It uses a) a neighborhood sampling strategy to increase efficiency, and b) a neutral network, a pooling layer, and an LSTM (Hochreiter and Schmidhuber, 1997) to aggregate the incoming messages instead of a simple sum. A drawback of GraphSage is, however, that it does not incorporate edge weights. The Graph attention network (GAT) of Veličković et al. (2017) introduced the attention mechanism into the GNN framework. This mechanism learns the relative importance of a node's neighbors, to assign more weight to the most important ones. Gilmer et al. (2017) presented the Message Passing Neural Network (MPNN) framework, unifying a large group of different GNN methods. Apart from the unifying framework, the most essential contribution of this model is in our view that it applies a learned message-passing function that utilizes edge features.

The aforementioned GNN methods have in common that their time complexity scales linearly with the number of edges in the graph (Wu et al., 2020). This property renders them well-suited for large, sparse graphs, which are characteristic of many real-world graphs.[3]

2.3.2. Heterogeneous GNN literature

Heterogeneous graph neural networks are commonly defined as extensions of existing homogeneous graph neural networks. For instance, the Relational Graph Convolutional Network (RGCN) (Schlichtkrull et al., 2018) extends the GCN framework to support graphs with multiple types of edges. RGCN achieves this by breaking down the heterogeneous graph into multiple homogeneous ones, one for each edge type. In each layer, GCN is applied to each homogeneous graph, and the resulting node embeddings are element-wise summed to form the final output. A drawback of RGCN is that it does not take node heterogeneity into account. Heterogeneous Graph Attention Networks (HAN) (Wang et al., 2019) generalize the Graph Attention Network (GAT) approach to heterogeneous graphs by considering messages between nodes connected by so-called meta-paths. Meta-paths (see the formal definition in Definition 2) are composite relationships between nodes that help to capture the rich structural information of heterogeneous graphs. HAN defines two sets of attention mechanisms. The first is between two different nodes, which is analogous to GAT. The second set of attention mechanisms is performed at the level of meta-paths, which computes the importance score of different composite relationships. Metapath Aggregated Graph Neural Network (MAGNN) (Fu et al., 2020) extends the approach of HAN by also considering the intermediary nodes along each meta-path (Dong et al., 2017). While HAN computes node-wise attention coefficients by only considering the features of the nodes at each end of the meta-path, MAGNN transforms all node features along the path into a single vector.

The interest in GNNs is rapidly growing, and advancements in this field are consistently being made. There are several other methods available that haven't been discussed here. For a more comprehensive understanding, we refer to the excellent survey conducted by Wu et al. (2020), which provides an overview of GNNs in general. Additionally, for insights specifically on Heterogeneous Network Representation Learning, including Heterogeneous GNNs, we refer to Yang et al., 2020 for a good overview.

3. Model

In this section we define and describe our proposed heterogeneous GNN model, which is based on the generic framework from Message Passing Neural Network (MPNN) introduced in Gilmer et al. (2017). We start by introducing the original (homogeneous) MPNN model and algorithm before we move on to give precise definitions for our heterogeneous graph setup and present our extension of the MPNN model for heterogeneous networks.

3.1. Message Passing Neural Network (MPNN)

Gilmer et al. (2017) introduces the generic MPNN framework which is able to express a large group of different GNN models, including GCN, GraphSage, and GAT. This is done by formulating the message passing with two learned functions, $M_k(\cdot)$, called the *message functions*, and $U_k(\cdot)$, called the *node updated functions*, with forms to be specified later. The framework runs K message passing iterations $k = 1, \ldots, K$ between nodes along the edges that connect them. The node representation vectors are initialized as their feature vectors, $\mathbf{h}_v^0 = \mathbf{x}_v$, and the previous representation $\mathbf{h}_v^{(k-1)}$ is the message that is being sent in each iteration k. After K iterations, the final

[3] This is also typically the case for graphs derived from bank transactions (Khazane et al., 2019), such as the graph that is the focus of this work.

representation $\mathbf{h}_v^{(K)}$ is passed on to an output layer to perform e.g. node-level prediction tasks. The message-passing function is defined as

$$\begin{aligned}\mathbf{m}_v^{(k)} &= \sum_{u \in N(v)} M_k(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{r}_{uv}),\\ \mathbf{h}_v^{(k)} &= U_k(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(k)}),\end{aligned} \tag{1}$$

where $N(v)$ denotes the neighborhood of node v, and $\mathbf{r}_{uv}$ represents the edge features for the edge between node u and v. By defining specific forms of $U_k(\cdot)$ and $M_k(\cdot)$, a distinct GNN method is formulated. From the viewpoint of this paper, an essential attribute of the MPNN framework is that the learned message-passing function utilizes edge features. Not many other proposed GNNs incorporate edge features into their model. Gilmer et al. (2017) emphasize the importance of edge features in the dataset they experiment on. For our dataset, the edge features contain essential information related to transactions and must be utilized in an expressive model.

3.2. Formal definitions

Before we can formulate our *heterogeneous* MPNN framework, we need to establish a precise definition of a heterogeneous graph, as well as a couple of additional concepts.

We largely adopt the commonly used graph notation from Wu et al. (2020), and use a heterogeneous graph definition which is a slight modification to those in Yang et al., 2020 and Wang et al. (2019) in order to allow for multiple edges of different types between the same two nodes.

Definition 1. (Heterogeneous graph) A heterogeneous graph is represented as $G = (V, E, \boldsymbol{X}, \boldsymbol{R}, Q^V, Q^E, \phi)$ where each node $v \in V$ and each edge e ∈ E has a type, and Q^V and Q^E denote finite sets of predefined node types and edge types, respectively. Each node $v \in V$ has a node type $\phi(v) = \nu \in Q^V$, where $\phi(\cdot)$ is a node type mapping function. Further, for $\phi(v) = \nu$, v has features $\boldsymbol{x}_v^\nu \in \boldsymbol{X}^\nu$, where $\boldsymbol{X}^\nu = \{\mathbf{x}_v^\nu | v \in V,\ \phi(v) = \nu\}$ and $\boldsymbol{X} = \{\mathbf{X}^\nu | \nu \in Q^V\}$. The dimension and specifications of the node feature $\mathbf{x}_v^\nu$ may differ for different node types ν. Further, let us denote by e_{uv}^ϵ an edge of type $\epsilon \in Q^E$ pointing from node u to v. Each edge e_{uv}^ϵ has features $\boldsymbol{r}_{uv}^\epsilon \in \boldsymbol{R}^\epsilon$, where $\boldsymbol{R}^\epsilon = \{\boldsymbol{r}_{uv}^\epsilon | u, v \in V\}$ and $\boldsymbol{R} = \{\boldsymbol{R}^\epsilon | \epsilon \in Q^E\}$. Just like for nodes, the edge features may have different dimensions for different edge types.

To formulate heterogeneous message passing, we will use the concept of *meta-paths*. Meta-paths are commonly used to extend methods from a homogeneous to a heterogeneous graph. For example, Dong et al. (2017) use meta-paths when introducing *metapath2vec*, which extends *DeepWalk* (Perozzi et al., 2014) and the closely related *node2vec* (Grover and Leskovec, 2016) to a method applicable on heterogeneous graphs. Wang et al. (2019) use meta-paths to generalize the approach of graph attention networks (Veličković et al., 2017) to that of heterogeneous graphs when formulating *heterogeneous graph attention network* (HAN). In addition to meta-path, the below definition introduces our own term, *meta-steps*, which we use when formulating our model.

Definition 2. (Meta-path, meta-step) A Meta-path belonging to a heterogeneous graph G is a sequence of specific edge types between specific node types,

$$(\nu_0, \epsilon_1, \nu_1, \epsilon_2, \ldots, \nu_{k-1}, \epsilon_k, \nu_k), \quad \nu_i \in Q^V, \epsilon_i \in Q^E.$$

here, k is the length of the meta-path. Let S be the set of meta-paths of length 1,

$$S = \{s = (\mu, \epsilon, \nu) | \mu, \nu \in Q^V, \epsilon \in Q^E\},$$

and refer to the elements s ∈ S as meta-steps.

We also introduce a definition of node neighborhood over a specific meta-step:

Definition 3. (Meta-step specific node neighborhood) Let $N_\mu^\epsilon(v)$ be the set of nodes of type μ which is connected to node v by an edge of type ϵ pointing to v:

$$N_\mu^\epsilon(v) = \{u \in V | \phi(u) = \mu, e_{uv}^\epsilon \in E\}.$$

We call $N_\mu^\epsilon(v)$ the (incoming) node neighborhood to node v with respect to the meta-step s = (μ, ϵ, ϕ(v)) ∈ S.

3.3. Heterogeneous MPNN

We are now ready to formulate our heterogeneous version of the MPNN method (HMPNN). The complete algorithm is provided in Algorithm 1. Our approach for extending a homogeneous GNN to a heterogeneous one closely follows the approach introduced by Schlichtkrull et al. (2018), where they generalize GCN to the method Relational Graph Convolutional Network (RGCN) applicable on graphs with multiple edge types.

The algorithm performs (at each iteration) multiple MPNN message passing operations, one for each meta-step $s \in S$ in the graph. Each of these has its separate learned functions $M_k^s(\cdot) = M_k^{(\mu,\epsilon,\nu)}(\cdot)$ and $U_k^s(\cdot) = U_k^{(\mu,\epsilon,\nu)}(\cdot)$, which allows the method to learn the context of each meta-step, and also allow message passing between nodes and across edges with varying numbers of features. The intermediate output of this process is multiple representation vectors for each node. To reduce these to a single vector, they are aggregated by a learned aggregation

function $A^{\nu}(\cdot)$ which is specific to each node type. In line with the generic formulation of MPNN we do not specify a specific form of the aggregation function in Algorithm 1. During the experiments, we have assessed two alternative options, which we will discuss shortly.

Our HMPNN model is implemented in Python, using the library PyTorch Geometric (PyG) (Fey and Lenssen, 2019) allowing for high-performance computing by utilizing massive parallelization through GPUs. Source code is available here: https://github.com/fredjo89/heterogeneous-mpnn https://github.com/hgnn20/heterogeneous-mpnn.

Algorithm 1 Heterogeneous MPNN

Initialize the representation of each node as its feature vector,

$$\boldsymbol{h}_v^{(0)} = \boldsymbol{x}_v^{\nu},\ \forall\, v \in V \text{ of type } \nu = \phi(v).$$

for $k = 1, \ldots, K$ **do** ▷ For each iteration

  for $v \in V$ **do** ▷ For each node

    for $\mu \in Q^V, \varepsilon \in Q^E$ such that $N_{\mu}^{\varepsilon}(v) \neq \emptyset$ **do** ▷ For each meta-step ending at $\phi(v)$

      Compute the new representation for node v of type $\nu = \phi(v)$, for the specific meta-step,

$$\boldsymbol{m}_v^{(\mu,\varepsilon,k)} = \sum_{u \in N_{\mu}^{\varepsilon}(v)} M_k^{(\mu,\varepsilon,\nu)}(\boldsymbol{h}_v^{(k-1)}, \boldsymbol{h}_u^{(k-1)}, \boldsymbol{r}_{uv}^{\varepsilon}),$$

$$\boldsymbol{h}_v^{(\mu,\varepsilon,k)} = U_k^{(\mu,\varepsilon,\nu)}(\boldsymbol{h}_v^{(k-1)}, \boldsymbol{m}_v^{(\mu,\varepsilon,k)}).$$

    end for

    Aggregate the representations from the multiple meta-steps into a single new representation for that node,

$$\boldsymbol{h}_v^{(k)} = A_k^{(\nu)}\left(\{\boldsymbol{h}_v^{(\mu,\varepsilon,k)} \mid \mu \in Q^V, \varepsilon \in Q^E\}\right)$$

  end for

end for

3.4. Choosing specific forms of the functions

As our setup is very general, the algorithm in Algorithm 1 gives rise to a whole range of new heterogeneous GNN methods. By defining specific forms of $U_k^{(\mu,\epsilon,\nu)}(\cdot)$, $M_k^{(\mu,\epsilon,\nu)}(\cdot)$ and $A_k^{(\nu)}(\cdot)$, a distinct GNN method is formulated. Note that there is nothing that prevents us from choosing different forms of these functions for different meta-steps or iterations. However, for the AML use case in Section 4 we have limited the scope to a single form for each of the three functions, respectively. These are described in the following.

We utilize the message function described in Gilmer et al. (2017):

$$M_k^{(\mu,\epsilon,\nu)}(\mathbf{h}_v^{(k-1)}, \mathbf{h}_u^{(k-1)}, \mathbf{r}_{uv}^{\epsilon}) = g_k^{(\mu,\epsilon,\nu)}(\mathbf{r}_{uv}^{\epsilon})\mathbf{h}_u^{(k-1)}.$$

here, $g_k^{(\mu,\epsilon,\nu)}(\cdot)$ is a single layer neural network which maps the edge feature vector $\mathbf{r}_{uv}^{\epsilon}$ to a $d^v \times d^u$ matrix, where d^u, and d^v are the number of features for the sending and receiving node type, respectively.

As update function we use

$$U_k^{(\mu,\epsilon,\nu)}\left(\mathbf{h}_v^{(k-1)}, \mathbf{m}_v^{(\mu,\epsilon,\nu,k)}\right) = \sigma\left(\mathbf{m}_v^{(\mu,\epsilon,k)} + \mathbf{B}_k^{(\mu,\epsilon,\nu)}\mathbf{h}_v^{(k-1)}\right),$$

where $\mathbf{B}_k^{(\mu,\epsilon,\nu)}$ is a matrix. Note that for a homogeneous graph, our choices of $M(\cdot)$ and $U(\cdot)$ are similar to those in Hamilton et al. (2017), except that the matrix applied in the message function is conditioned on the edge features rather than being the same across all edges.

For the aggregation function, we consider two alternatives. The first is to take the sum of the vectors from each meta-step before performing a nonlinear transformation, in the same fashion as (Schlichtkrull et al., 2018):

$$A_k^{(\nu)}\left(\left\{\mathbf{h}_v^{(\mu,\epsilon,k)} \middle| \mu \in Q^V, \epsilon \in Q^E\right\}\right) = \sigma\left(\sum_{\mu \in Q^V, \epsilon \in Q^E} \mathbf{h}_v^{(\mu,\epsilon,k)}\right). \tag{2}$$

Here, $\sigma(\cdot)$ is the sigmoid function. In the second aggregation alternative, the vectors $\mathbf{h}_v^{(\mu,\epsilon,k)}$ are concatenated into a single vector. A single-layer perceptron (neural network) is then applied to it, and outputs the new representation:

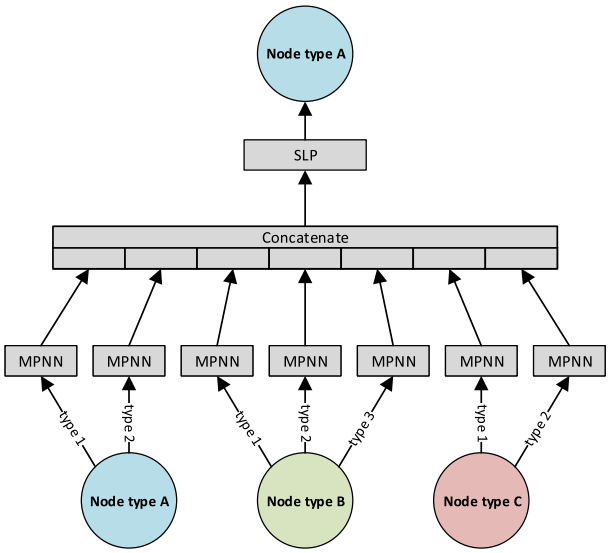

Fig. 2. Illustration of the architectural extension of the homogeneous MPNN model to our HMPNN model (HMPNN-ct) as messages are passed to one of the node types (A), from three node types (A, B, and C) and with three edge types (1, 2 and 3). SLP is short for Single Layer Perceptron (neural network). Analogous architectures are used for messages passed to the other node types.

$$A_k^{(\nu)}\left(\left\{\mathbf{h}_v^{(\mu,\epsilon,k)} \middle| \mu \in Q^V, \epsilon \in Q^E\right\}\right) = \sigma\left(W_k^{(\nu)} \underset{\mu\in Q^V,\epsilon\in Q^E}{\|} \sigma\left(\mathbf{h}_v^{(\mu,\epsilon,k)}\right)\right). \tag{3}$$

We denote the two resulting models *HMPNN-sum* and *HMPNN-ct*, respectively. Fig. 2 illustrates the architectural extension of the homogeneous MPNN model to our HMPNN model (HMPNN-ct) as messages are passed to one of the node types, where (3) is used as aggregation function.

4. AML use case

In this section, we first describe the heterogeneous graph data. Then, we lay out the setup of the experiments we have performed on this dataset, before we provide the results.

4.1. Data

The graph is established based on customer and transaction data from Norway's largest bank, DNB, in the period from February 1, 2022, to January 31, 2023. The nodes in the graph represent entities that are senders and/or recipients of financial transactions. If two entities participate in a transaction with each other, this is represented by an edge (of type transaction) between the two, where the direction of the edge points from the sender to the recipient. There are in total about 5 million nodes and 10 million such edges in the graph.

There are three types of nodes in the graph. The first one is called *individual* and represents a human individual's customer relationship in the bank. It includes all of the individual's accounts in the bank.[4] The second type of node is called *organization* and represents an organization's or company's customer relationship in the bank in the same manner as a node representing an individual. The third type of node is called *external* and represents a sender or recipient of a transaction that is outside of the bank. The three node types have separate sets of features that contain basic information about the entity. Table 1 shows the number of nodes and intrinsic features for each of the three types.

The majority of the edges in the graph represent presence of a financial transaction between different individuals/organizations/external entities in the edge direction. In addition to this edge type, the graph includes role as a second edge type. That edge points from an individual to an organization if the individual occupies a position on the board, is the CEO, or holds ownership in the organization. The resulting graph is directed and heterogeneous with respect to both nodes and edges. Fig. 3 shows the schema of the graph, including the nine possible meta-steps. As shown in the schema, there are no edges between different external nodes. The reason for this is that the bank lacks access to transactions that do not involve their customers. The transaction edges have as features the number and monetary amount of transactions made within the one-year period. The role edges have two features, the first denoting the

[4] Transactions made to/from any of the accounts in the bank belonging to the customer will result in an edge to/from the node representing the individual.

Table 1
Number of nodes and features for the three types of nodes in the graph.

Type	#Nodes	#Features
Individuals	1,718,159	11
Organizations	204,770	8
Externals	3,226,230	2
Total number of nodes	5,149,159	–

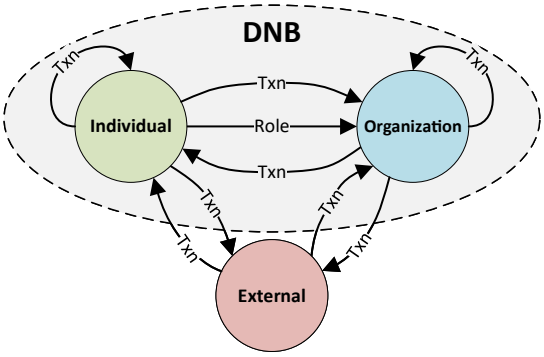

Fig. 3. The schema of the graph that has been the subject of our experiments, with the nodes representing customer relationships in DNB outlined by the gray ellipse. Here, the transaction edges are abbreviated as *Txn*.

role type and the second the ownership percentage (provided the role represents ownership in the organization). The number of edges for each meta-step is listed in Table 2.

The nodes that represent individuals are assigned a binary class (0 for regular individuals, and 1 for individuals known to conduct suspicious behavior). As mentioned in the introduction, the data only contains labels for individual nodes. Suspicious individuals are defined as those that have been subject to an AML case (stage 2 in Fig. 1) during a certain time window. Note that customers implicated in cases that were not reported to the FIU are still defined as suspicious. This choice was made because our objective is to model suspicious activity, which these customers certainly have conducted, even though the suspiciousness was diminished by a close manual inspection. In total, 7732 individuals are labelled as class 1, which amounts to 0.45% of all the individual nodes in the dataset.

Due to the sensitive nature of these data, containing both personal and possibly competition sensitive information for the bank, the data are not shareable. We are neither allowed to reveal further details of the graphs nor the exact features associated with the different nodes in our model. Below, we give a broad overview of the characteristics and features of the graph, within our permission restrictions. To get a feeling of the local characteristics of the graph, Fig. 4 shows egonets of four random nodes, with the starting node enlarged. The upper and lower panels show, respectively, the 3-hop and 9-hop egonets of the (undirected) transaction and role edges. The number of shown hops was chosen to balance presentability and amount of detail. Moreover, Fig. 5 shows histograms of the degree centrality for the three different node types.

Table 2
Number of instances for each meta-step in the graph. Here, the transaction edges are abbreviated as *txn*, and the node types Individual, Organization and External are abbreviated *ind*, *org* and *ext*, respectivly.

Type	#Edges
ind-txn-ind	1,532,264
ind-txn-org	361,444
ind-txn-ext	2,800,249
org-txn-ind	359,180
org-txn-org	368,928
org-txn-ext	990,441
ext-txn-ind	2,237,038
ext-txn-org	632,690
ind-role-org	920,716
Total number of edges	10,202,950

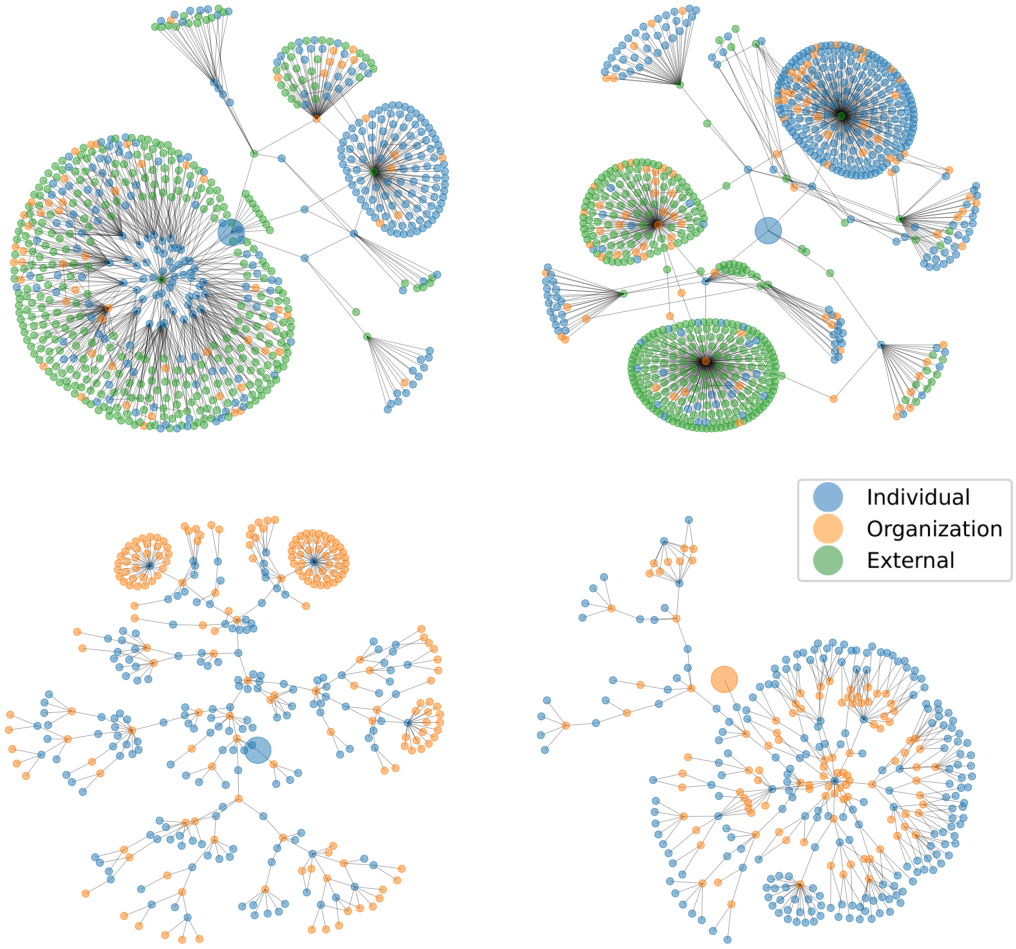

Fig. 4. Homogeneous egonets for four random nodes in the graph. The upper two panels show 3-hop egonets of the (undirected) transaction edges in the graph. The lower two panels show 9-hop egonets for the (undirected) role edges in the graph. The starting node is enlarged.

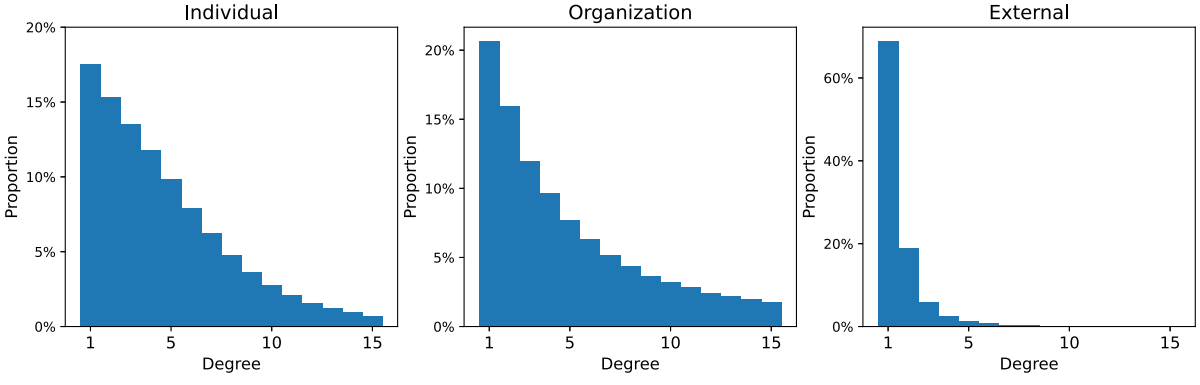

Fig. 5. Histogram showing the distribution of the degree centrality for the three different node types, completely ignoring the edge type.

Some nodes have a large number of neighbors, while others have few. While the degree distribution of individuals and organizations is similar, the distribution for organizations has a thicker tail, indicating that it's more common for organizations to exhibit a higher degree. As we don't have knowledge of edges between external nodes, this node type typically has much fewer neighbors.

4.2. Experiment setup

The goal of this use case and experiment is to predict the label (suspicious/regular) on nodes where the label is unknown to the model. As mentioned, we concentrate on building models for detecting fraudulent *individual* customers, i.e. only the individual nodes are labeled. To mimic a realistic scenario with partially observed labels, we split the set of *individual* nodes into a training set and a test set. While the full graph is used for message passing in both training and inference, only the labels of individuals in the training set are available during model training. At test time, predictions and evaluations are performed exclusively on the individuals in the held-out test set, whose labels are not used during training.

We used an 80/20% train-test split, where the splitting was performed using stratified random sampling with allocation proportional to the original class balance to preserve the class balance between the two sets. This resulted in 6185 suspicious individuals in the training set and 1547 in the test set. The same split was used for all methods.

To reflect real AML deployment conditions, we did not modify the class distribution during the training of any model. Preserving the real class balance maintains the operational realism of the dataset and allows model outputs to be interpreted as probabilities under the true data-generating process. Rebalancing techniques such as class reweighting or loss adjustments typically change the effective class prior and complicate the interpretation of model outputs.

In theory, a random train–test split may introduce a degree of temporal leakage. However, in our setting we have multiple reasons to believe this effect is limited. Customers are not informed that they have been reported, which reduces the scope for behavioral feedback. In addition, suspicious activity typically unfolds over longer time periods rather than as isolated short-term events. Finally, the graph only contains customers who were active at the time of data extraction. For these reasons, we believe that the risk of meaningful forward peeking is small in this scenario.

To evaluate the performance of the HMPNN-models, we benchmark and compare their performance to three non-graph methods supplied with additional node features, and one additional GNN method. The models are applied with multiple model complexity configurations. To enhance the non-graph methods, we generated 83 additional node features that accompany the original node features. These include 11 summaries of the node-neighborhoods, i.e. the number of neighbors of different types, both incoming and outgoing. We also computed 8 weighted summaries of node-neighborhoods, specifically for transaction edges and the node feature monetary amount. Additionally, we added 64 features generated using metapath2vec (Dong et al., 2017). These were assembled from embeddings of dimension 8 generated for each meta-path of length 2 starting and ending at a node of type individual. Together with the 11 intrinsic node features, this results in a total of 94 features. For more detailed information on these additional node features, please refer to Appendix A. The three benchmark methods are briefly described below:

XGBoost This gradient-boosted tree method is included as a strong non-relational baseline. By combining many shallow trees in a sequential boosting process, it effectively captures complex nonlinear patterns and interactions, and has previously shown strong performance on AML classification tasks in Jullum et al., 2020.

Logistic regression This classic method serves as a very basic benchmark not directly utilizing the network information and with a basic and inflexible parametric form.

Regular Neural Network This method, applied with 1 and 2 hidden layers, is much more flexible than the logistic regression model, but neither utilizes the network directly.

HGraphSage GraphSage (Hamilton et al., 2017) is a well-known homogeneous GNN method and is applied to our heterogeneous graph in the same fashion as HMPNN, as described below. In contrast to MPNN, GraphSage does not utilize edge features. We, therefore, test its performance both with and without additional node features that hold the weighted in/out degrees for the transaction edges, and the weight is the transaction amount on the edges. This results in 6 additional node features for nodes of type *individual* and *organization*, and 4 on nodes of type *external*. We test the method with the aggregation function in (2) (HMPNN-sum). We applied the method with 0, 1 and 2 hidden layers.

The XGBoost, Logistic regression and Regular Neural Network model, all have access to the additional network-generated node features. For the HMPNN-models, described in Section 3.3, we test the method with the two aggregation functions in (2) (HMPNN-sum) and (3) (HMPNN-ct). We applied the method with 0, 1, and 2 hidden layers for each of the aggregation methods. In total, our experiment contains 16 different models/method variants. The number of parameters involved in each of these is listed in Table D.6 in the Appendix.

The XGBoost model was trained in R using the mlr3 package. Following the ensemble strategy used in Jullum et al., 2020, we use a CV-mean ensemble model: the data are split into four folds, and separate XGBoost models are trained on each 3/4 split, using the left-out fold for early-stopping validation. The final model prediction is obtained by averaging the outputs of the four cross validation models, which stabilizes the predictions and reduces variance relative to using a single model. Some details on the selection of hyperparameters are provided in Appendix B.

The Logistic Regression and Regular Neural Network models were trained using the open source deep learning library PyTorch (Paszke et al., 2019). Architectural choises for Regular Neural Network are provided in Appendix C.

HMPNN and HGraphSage were trained using the open source library PyTorch Geometric (PyG), which expands PyTorch with utilities for representing and training GNNs.

Logistic regression, regular neural networks, and the GNN models were all trained with the Adam optimizer (Kingma and Ba, 2014), using the Binary Cross Entropy loss function:

$$\text{Loss}(\hat{y}_v, y_v) = -(y_v \cdot \log(\hat{y}_v) + (1 - y_v) \cdot \log(1 - \hat{y}_v)).$$

The hyperparameters for each of the methods were tuned using 5-fold cross-validation on the training set. Here, three hyperparameters were determined: (1) The regularization strength, (2) the learning rate, and (3) the number of training iterations, i.e., by early stopping. For all methods, the learning rate was in the range $[10^{-4}, 10^{-1}]$. As for regularization, the L_2-constraint was used, and was in the range $[10^{-8}, 10^{-1}]$. The optimal value for the L_2 constraint was highly dependent on the complexity of the model to be trained.

The experiments were carried out using *Python 3.7.0*, with PyTorch 1.12.1 and PyG 2.2.0. The computer used to run the experiments had 8 CPUs of the type *High-frequency Intel Xeon E5-2686 v4 (Broadwell) processors*, with 61 GB shared memory, and one GPU of type *NVIDIA Tesla V100* with 16 GB memory. This GPU has 5120 CUDA Cores and 640 Tensor Cores.

4.3. Results

For each node in the test set, all the different methods output a score between 0 and 1, reflecting the probability that the node is a suspicious customer. To measure the performance of the different methods on the test set, we use three different types of evaluation metrics motivated by the characteristics of our task and data. The overall performance is measured by the area under the precision/recall curve (PR AUC) and the area under the receiver operator curve (ROC AUC). PR AUC computes the area under the curve obtained by plotting the *precision* (TP/(TP + FP)) as a function of the *recall* (TP/(TP + FN)), and PR AUC computes the area under the curve obtained by plotting the recall as a function of *False Positive Rate* (FP/(FP + TN)). Here TP/FP/TN/FN represents the number of classified nodes which are, respectively, true positives, false positives, true negatives, and false negatives. PR AUC is our primary overall evaluation metric due to its suitability for severe class imbalance, while we also report ROC AUC as a widely used, threshold-independent measure. We also report precision at fixed recall levels, as this has a direct operational interpretation in AML settings and is easier to relate to real-world decision-making.

Fig. 6 displays the ROC AUC and PR AUC on the test set for all different methods and number of hidden neural network layers used by the respective methods.

All methods benefit from including more layers. HMPNN-ct with 2 hidden layers is the best method in terms of both PR AUC and ROC AUC. HMPNN-ct does quite well also when applied without hidden layers, while the other network models require more layers to achieve a decent performance. This indicates an effective network arcitecture in the HMPNN-ct model. Note also that, the Logistic Regression (Regular Neural Network with 0 hidden layers) performs quite well in terms of both ROC AUC and PR AUC considering the that it does not know anything about the network, except for additional network summary features, and has the simplest form of architecture. Note, however, that the Regular Neural Networks only do slightly better than Logistic Regression. This indicates that the simple architecture of the Logistic Regression model is not a significant downside for that limited dataset. The XGBoost model performs somewhat better than both Logistic Regression and the Regular Neural Networks, especially at the lowest recall levels, highlighting the benefit of its nonlinear boosting framework, even without exploiting the underlying graph. Still, its performance remains clearly below that of the GNN models (with at least one hidden layer), illustrating the additional predictive power gained from modeling the relational structure directly. Thus, we believe that the lack of performance for the other network models is related to an inappropriate and inefficient architecture compared

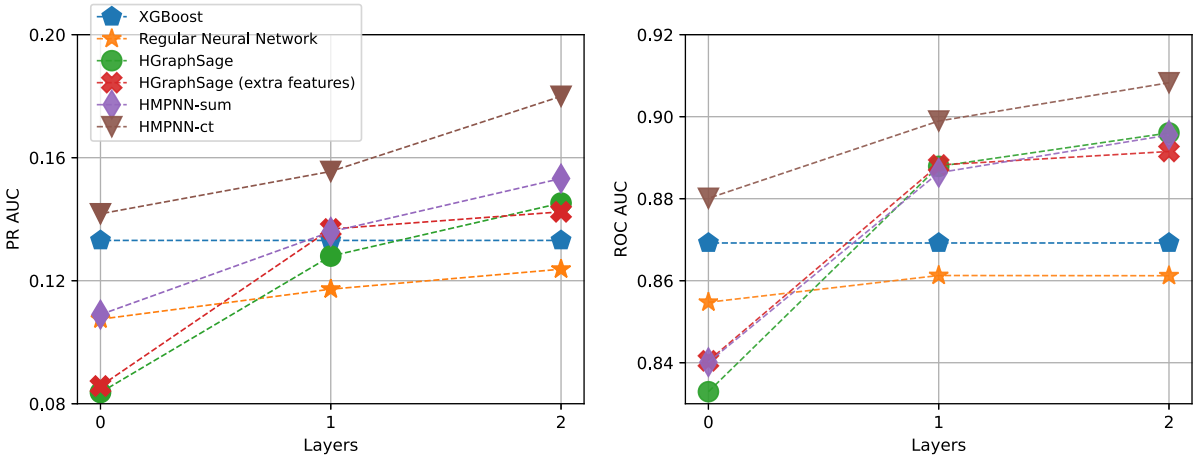

Fig. 6. ROC AUC and PR AUC on the test set for all different methods and number of hidden neural network layers used by the methods. The Regular Neural Network with 0 layers corresponds to Logistic Regression. The XGBoost baseline model is shown as horizontal lines the two panels.

Table 3
Precision at specific values of Recall for the different models. For completeness, PR AUC and ROC AUC for also listed.

Model	Number of	Recall(%)				PR	ROC
	Hidden Layers	1	5	10	50	AUC	AUC
XGBoost	–	66.67	50.00	36.90	6.20	0.1331	0.8692
Regular	0	61.54	36.28	28.55	5.53	0.1075	0.8547
Neural Network	1	64.00	43.82	30.51	5.89	0.1173	0.8613
	2	61.54	47.56	34.68	5.88	0.1237	0.8612
HGraphSage	0	59.26	33.62	20.58	4.01	0.0836	0.8329
	1	61.54	48.15	38.56	6.51	0.1280	0.8879
	2	59.26	60.47	42.47	7.19	0.1452	0.8960
HGraphSage	0	48.48	34.21	21.29	4.24	0.0858	0.8405
(extra features)	1	64.00	50.65	35.39	7.41	0.1368	0.8882
	2	69.57	50.32	38.27	7.68	0.1424	0.8915
HMPNN-sum	0	64.00	38.24	29.75	5.34	0.1090	0.8401
	1	69.57	42.62	36.38	7.76	0.1359	0.8863
	2	84.21	53.79	38.27	8.67	0.1532	0.8955
HMPNN-ct	0	76.19	48.75	38.18	6.92	0.1418	0.8801
	1	61.54	50.65	43.66	8.96	0.1555	0.8989
	2	66.67	58.21	50.99	10.25	0.1800	0.9083

to that of HMPNN-ct. The fact that overall, HMPNN-sum does not perform on par with HMPNN-ct further indicates that the performance boost in HMPNN-ct is mainly due to the architectural trick of the last single-layer neural network.

Considering the limitations in resources faced by banks, conducting thorough examinations of a substantial volume of suspicious cases is typically unfeasible. Therefore, the primary purpose of the model is to generate a limited set of high-quality predictions where money laundering is likely to occur, meaning that the precision at small to medium-sized recall levels is more relevant than the higher ones. Table 3 shows the precision corresponding to recall levels of 1%, 5%, 10%, and 50%, respectively, and allows studying the performance of the methods in greater depth and at a wider range.

Focusing on HMPNN-ct, we see that when the classification threshold is set such that we identify 1% of the suspicious customers (recall $= 1\%$) two-thirds of those classified as suspicious *are* actually suspicious (precision $\approx 67\%$). Increasing the classification threshold to 5% or 10% gives precisions of about 58% and 51%. Moreover, if we decrease the threshold such that half of the suspicious customers (recall $= 50\%$) are identified, 90% of the customers classified as suspicious are not really suspicious. These rates may not seem impressive at first glance. However, considering the severe class imbalance in the data (0.45% of the total number of observations are suspicious), and the fact that detection of money laundering is a notoriously difficult problem, these performance scores are very promising.

Finally, note that even though the 2-layer HMPNN-sum model performs worse than HMPNN-ct overall and for the larger recalls, it obtains a significantly better precision (84% vs 67%) at recall 1%. In essence, this model is better at detecting the most evident instances of money laundering. This aspect is crucial to consider when selecting a model, particularly if resource limitations restrict the investigation to a small number of customers for potential money laundering.

5. Summary and concluding remarks

The purpose of the present paper is to model and detect suspicious behavior related to money laundering within a large-scale real-world heterogeneous graph. The graph is extracted from a realistic AML scenario in Norway's largest bank, encompasses 5 million nodes and 10 million edges, and comprises customer data, transaction data, and business role data. We adopt a GNN approach to model these data, extending the (homogeneous) MPNN architecture of Gilmer et al. (2017) to accommodate our heterogeneous setting. This modified architecture, termed HMPNN (Heterogeneous Message Passing Neural Network), employs distinct message-passing operators for each combination of node and edge types to effectively manage the graph's heterogeneity. Two versions of the model are proposed: HMPNN-sum and HMPNN-ct. HMPNN-ct constructs the final node embeddings by concatenating each node-edge operator into a single-layer neural network, while HMPNN-sum uses the simpler approach of summing the embeddings from the node-edge operators. Overall, HMPNN-ct outperformed HMPNN-sum as well as all alternative models by a significant margin. HMPNN-sum was, however, the best model at recall $= 1\%$, i.e. it was most accurate for the customers assigned the largest probabilities of being suspicious. In particular, all HMPNN variants with hidden layers consistently outperform the non-graph baselines, including the XGBoost ensemble model, which has previously shown strong performance on AML classification tasks (Jullum et al., 2020). This underscores the benefit of explicitly modeling relational information in AML networks.

As we saw from the overall measures in Fig. 6, all models performed the best when fitted using 2 hidden layers. We may have gotten even better performance if we increased the number of layers further. However, 2 layers is where we hit the memory roof on our GPU, so we were unable to explore this in practice. It is worth noting that the HMPNN-ct architecture is also clearly the most successful when we restrict the number of hidden layers to 1 or 0. This is relevant as larger networks, and/or less computational resources or training time available, may in other situations demand a reduction in the number of layers. From Table D.6 in the Appendix, we also see that

apart from the regular neural network model, HMPNN-ct has the fewest number of model parameters of the 2-layer models, indicating that it has an efficient architecture.

To provide context for the performance of our models, consider the following two applications within a bank's surveillance system: With a cutoff threshold at 1% recall, our models can generate accurate alerts for a small number of highly suspicious customers. Our results suggest that these alerts can achieve an accuracy of up to 84%. For comparison, PwC (2010); Bakry et al. (2023); Kalra (2023) indicate accuracy rates between 3% and 10% in traditional surveillance systems. Additionally, because our model leverages relational data, it is likely to address weaknesses in traditional surveillance systems that rely solely on entity-based information. Integrating our model could therefore enhance overall performance and close gaps in the surveillance system. With a cutoff threshold at 50% recall, our models can be used to categorize customers into high-risk and low-risk segments. Our results suggest that this approach would result in a high-risk segment comprising 2.2% of the total number of customers, capturing 50% of the known suspicious customers. Conversely, the low-risk segment would include the remaining 97.8% of customers and 50% of known suspicious customers. This segmentation could be used to perform more frequent and rigorous due diligence for those in the high-risk segment.

When implementing a predictive model for suspicious transactions within a real AML system, several crucial decisions need to be made. One vital consideration involves determining the optimal stage in the process (see Fig. 1) for applying the predictions: either preceding the alert inspection or preceding the case investigation. If the predictions are used prior to the alert inspection, it might be preferable to set a classification threshold with a higher recall. On the other hand, if the predictions are applied before the case investigation, it may be more appropriate to select a more stringent threshold, i.e. that has a lower recall. This would help minimize the allocation of investigation resources towards false positives, leading to greater efficiency.

It's crucial to recognize that customers labeled as "regular" may, in fact, be suspicious and potentially involved in money laundering – they simply haven't been controlled in the existing AML system and are therefore labeled as "regular". This scenario holds true for practically all instances of money laundering modeling. Consequently, if a customer labeled as "regular" is assigned a high probability of being suspicious by the model, it is possible that the customer has been mislabeled. As a result, modeling test phases with a labeled test set, as outlined here, can also serve as a means to generate suggestions for customers who warrant further investigation into their past behaviors. In other words, the modeling approach allows for the identification of customers who may require re-examination based on the model's predictions, even if they were initially labeled as "regular".

In order to increase the performance of our money laundering modeling approach, some aspects become readily apparent. While our dataset is rich in terms of the number of nodes and edges, contains network data from both financial transactions and professional roles, and has a large number of edge and node features, it can always be richer. In our setup, the transaction edges contain the number and total monetary amount made in the one-year period. This could be refined by also including the standard deviation, median, and other summary statistics like in Jullum et al., 2020. Moreover, provided high-quality data can be obtained, it would be valuable to include customer links using connections from social media platforms, geographical information such as shared addresses or phone numbers, or even family relations.

As mentioned, organizational customers are fewer in number and exhibit less homogeneity compared to individuals, rendering them less suitable for modeling compared to individuals. It still presents a natural candidate for further work to develop models that make predictions on the bank's organization customers. However, to truly see the potential of such a model, we believe that it is essential to expand the dataset in time to include more fraudulent organizations to learn from and enrich the data with more organization-specific features.

A significant limitation that applies to our work, as well as most endeavors related to money laundering detection, is the restricted nature of the data, which is confined to customers within a single bank. Although our graph includes "external" customers from other banks, the transactions and professional role links between customers in external banks are unavailable. This is primarily due to banks being either unwilling or prohibited from merging their customer information with other banks, due to security regulations or competitive considerations. Surmounting these data-sharing challenges among prominent financial institutions would enable the modeling of a more comprehensive network of transactions and relationships, leaving fewer hiding places for money launderers. Nevertheless, the administration of such collaborative, analytical, and modeling systems demands substantial resources and investments, a process that may take years to accumulate.

In any case, to the best of our knowledge, no scientific work has been published regarding the utilization of heterogeneous graph neural networks in the context of detecting money laundering on a large-scale real-world graph. This paper should be viewed as a first attempt to leverage heterogeneous GNN architecture within AML and has showcased promising outcomes. We envision that our paper will provide invaluable perspectives and directions for scholars and professionals engaged in money laundering modeling. Ultimately, we hope this contribution will aid in the continuous endeavors to combat money laundering – a persistent and pressing issue in the global financial economy.

Code availability

The implementation of our HMPNN model is available through the following repository: https://github.com/fredjo89/heterogeneous-mpnn.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: One of the authors (Fredrik Johannessen) was employed by DNB during the years 2018–2023. Parts of this work were carried out during that period.

Acknowledgements

This work was supported by the Research Council of Norway, grant number 237718. We would like to thank Daniel Piacek for his assistance running the XGBoost baseline experiments. We are also grateful to DNB for providing access to the real-world data used in this study and for supporting the publication of this research. Finally, we thank the Editor-in-Chief Jingzhi Huang and the two anonymous reviewers for their valuable feedback and contribution to the revision of this manuscript.

Appendix A. Network features

This appendix provides details of how we created additional node features that capture information about a node's role in the network. These were utilized in our entity-based models (and HGraphSage) for benchmarking in Section 4.

We generated a total of 83 additional features, which when combined with the original 11 intrinsic node features, resulted in a total of 94 features. These additional features are categorized into three distinct types: 1) Unweighted Neighborhood summary consisting of 11 features, 2) Weighted Neighborhood summary consisting of 8 features, and 3) Metapath2vec embeddings with a total of 64 features.

Unweighted Neighborhood Summary. The unweighted neighborhood summary features encapsulate the (unweighted) in/out degree for each meta-step that nodes of type individual are part of, see Fig. 3. This amounts to seven features, six from transaction edges and one from role edges. Further, four summary features are added: 1) The total in-degree, which is the count of all incoming edges, irrespective of the meta-step, 2) The total out-degree, which is the count of all outgoing edges, irrespective of the meta-step, 3) The total degree, which is the count of all edges, irrespective of the meta-step or the direction, 4) The total count of meta-steps in which the node is involved. In total, this gives 11 additional node features.

Weighted Neighborhood Summary. The weighted neighborhood summary features contain the weighted in/out degree for each meta-step involving transaction edge. Here the edge feature representing the monetary amount was used as edge weight. This provides six features for a node of type individual. Further, two summary features are added: 1) The total weighted in-degree, 2) The total weighted out-degree, In total, this gives 8 additional node features.

Metapath2vec Embeddings. Metapath2Vec was used to generate embeddings for each of the four meta-paths shown in Table A.4. These are all meta-paths of length 2 that start and end at a node of type individual. The dimension of the embedding for each meta-path was set to 8. The embeddings were added as features on both the node at the start and end of the meta-path. This amounts to 64 additional node features. We used the implementation of MetaPath2Vec in PyTorch Geometric. Table A.5 lists the parameters used in the creation of the embeddings.

Table A.4
Meta-paths for Individuals

$\texttt{ind} \xrightarrow{\texttt{txn}} \texttt{ind} \xrightarrow{\texttt{txn}} \texttt{ind}$
$\texttt{ind} \xrightarrow{\texttt{txn}} \texttt{org} \xrightarrow{\texttt{txn}} \texttt{ind}$
$\texttt{ind} \xrightarrow{\texttt{txn}} \texttt{ext} \xrightarrow{\texttt{txn}} \texttt{ind}$
$\texttt{ind} \xrightarrow{\texttt{role}} \texttt{org} \xrightarrow{\texttt{txn}} \texttt{ind}$

Table A.5
Hyperparameters for the creation of MetaPath2Vec-embeddings.

Parameter	Value
embedding_dim	8
walk_length	20
context_size	10
walks_per_node	10
num_negative_samples	1

Appendix B. XGBoost hyperparameters

The hyperparameters of the XGBoost model were tuned using a simple random search procedure implemented via the mlr3 framework in R. Candidate models were evaluated using 4-fold cross-validation where early stopping was applied for each of the four models based on the PR AUC on the hold-out fold.

The best-performing configuration was found with the following hyperparameters: alpha = 0.0003, colsample_bytree = 2.69, eta = 0.032, gamma = 0.030, lambda = 14.60, max_depth = 10, min_child_weight = 24.80, subsample = 0.68. The four cross-validated models used in the CV-mean ensemble contained, respectively, 368, 378, 320, and 362 boosting rounds (trees).

Appendix C. Regular Neural Network Architecture

The baseline neural network was designed with the number of neurons in each hidden layer matching the number of input features (94 units), which provided a natural starting point. We experimented with a range of alternative configurations, including different numbers of hidden layers, varying layer widths, and multiple activation functions. None of these variations produced more than a negligible improvement in predictive performance. Other architectural options such as dropout layers and skip connections were not explored.

The model complexity was also constrained by available GPU memory. However, given the small performance difference observed between networks with one and two hidden layers, there is little evidence that deeper architectures would provide significant gains for this task.

Appendix D. Network parameters

Table D.6 shows the number of parameters for each of the models discussed in Section 4.

Table D.6
Number of model parameters for the different models

Model	Number of Hidden Layers		
	0	1	2
NeuralNetwork	95	9,025	17,955
HGraphSage	189	3,999	7,809
HGraphSage (extra features)	329	13,045	25,761
HMPNN-sum	296	6,536	12,776
HMPNN-ct	3,071	4,487	6,303

References

Alarab, I., Prakoonwit, S., 2022. Graph-based LSTM for anti-money laundering: experimenting temporal graph convolutional network with bitcoin data. Neural Processing Letters 1–19.

Alarab, I., Prakoonwit, S., Nacer, M.I., 2020. Competence of graph convolutional networks for anti-money laundering in bitcoin blockchain. In: Proceedings of the 2020 5th International Conference on Machine Learning Technologies, pp. 23–27.

Bai, Y., Ding, H., Bian, S., Chen, T., Sun, Y., Wang, W., 2019. Simgnn: a neural network approach to fast graph similarity computation. In: Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining, pp. 384–392.

Bakry, A.N., Alsharkawy, A.S., Farag, M.S., Raslan, K., 2023. Automatic suppression of false positive alerts in anti-money laundering systems using machine learning. The Journal of Supercomputing 1–21.

Bartolozzi, D., Gara, M., Marchetti, D.J., Masciandaro, D., 2022. Designing the anti-money laundering supervisor: the governance of the financial intelligence units. International Review of Economics & Finance 80, 1093–1109.

Bjerregaard, E., Kirchmaier, T., 2019. The Danske Bank Money Laundering Scandal: a Case Study. Available at: SSRN 3446636.

Bolton, R.J., Hand, D.J., 2002. Statistical fraud detection: a review. Statistical Science 17, 235–255.

Chami, I., Abu-El-Haija, S., Perozzi, B., Ré, C., Murphy, K., 2022. Machine learning on graphs: a model and comprehensive taxonomy. Journal of Machine Learning Research 23, 1–64.

Chen, J., Ma, T., Xiao, C., 2018a. FastGCN: fast learning with graph convolutional networks via importance sampling. In: International Conference on Learning Representations. International Conference on Learning Representations, ICLR.

Chen, Z., Van Khoa, L.D., Teoh, E.N., Nazir, A., Karuppiah, E.K., Lam, K.S., 2018b. Machine learning techniques for anti-money laundering (aml) solutions in suspicious transaction detection: a review. Knowledge and Information Systems 57, 245–285.

Colladon, A.F., Remondi, E., 2017. Using social network analysis to prevent money laundering. Expert Systems with Applications 67, 49–58.

de Jesús Rocha-Salazar, J., Segovia-Vargas, M.J., del Mar Camacho-Miñano, M., 2021. Money laundering and terrorism financing detection using neural networks and an abnormality indicator. Expert Systems with Applications 169, 114470.

Deng, X., Joseph, V.R., Sudjianto, A., Wu, C.J., 2009. Active learning through sequential design, with applications to detection of money laundering. Journal of the American Statistical Association 104, 969–981.

Deprez, B., Vanderschueren, T., Baesens, B., Verdonck, T., Verbeke, W., 2025. Network analytics for anti-money laundering—a systematic literature review and experimental evaluation. Informs Journal on Data Science 0 (0). https://doi.org/10.1287/ijds.2024.0042.

Dong, Y., Chawla, N.V., Swami, A., 2017. metapath2vec: scalable representation learning for heterogeneous networks. In: Proceedings of the 23rd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 135–144.

Elliott, A., Cucuringu, M., Luaces, M.M., Reidy, P., Reinert, G., 2019. Anomaly detection in networks with application to financial transaction networks. arXiv preprint arXiv:1901.00402.

Fey, M., Lenssen, J.E., 2019. Fast graph representation learning with pytorch geometric. arXiv preprint arXiv:1903.02428.

Fruth, J., 2018. Anti-Money Laundering Controls Failing to Detect Terrorists, Cartels, and Sanctioned States. https://www.reuters.com/article/bc-finreg-laundering-detecting-idUSKCN1GP2NV. (Accessed 14 July 2022).

Fu, X., Zhang, J., Meng, Z., King, I., 2020. Magnn: metapath aggregated graph neural network for heterogeneous graph embedding. In: Proceedings of the Web Conference 2020.

Garin, L., Gisin, V., 2023. Machine learning in classifying bitcoin addresses. The Journal of Finance and Data Science 9, 100109. https://doi.org/10.1016/j.jfds.2023.100109. URL: https://www.sciencedirect.com/science/article/pii/S2405918823000259.

Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E., 2017. Neural message passing for quantum chemistry. In: International Conference on Machine Learning. PMLR, pp. 1263–1272.
Grover, A., Leskovec, J., 2016. node2vec: scalable feature learning for networks. In: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 855–864.
Hamilton, W.L., Ying, R., Leskovec, J., 2017. Inductive representation learning on large graphs. In: Proceedings of the 31st International Conference on Neural Information Processing Systems, pp. 1025–1035.
Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Computation 9, 1735–1780.
Hu, Z., Dong, Y., Wang, K., Sun, Y., 2020. Heterogeneous graph transformer. In: Proceedings of the Web Conference 2020.
Jullum, M., Løland, A., Huseby, R.B., Ånonsen, G., Lorentzen, J., 2020. Detecting money laundering transactions with machine learning. Journal of Money Laundering Control 23 (1), 173–186.
Kalra, B.B., 2023. In 2023, necessity will drive banking innovation. Global Banking & Finance Review. URL: https://www.globalbankingandfinance.com/in-2023-necessity-will-drive-banking-innovation/. (Accessed 25 June 2024).
Khazane, A., Rider, J., Serpe, M., Gogoglou, A., Hines, K., Bruss, C.B., Serpe, R., 2019. Deeptrax: embedding graphs of financial transactions. In: 2019 18th IEEE International Conference on Machine Learning and Applications (ICMLA). IEEE, pp. 126–133.
Kingma, D.P., Ba, J., 2014. Adam: a method for stochastic optimization. arXiv preprint arXiv:1412.6980.
Kipf, T.N., Welling, M., 2016a. Semi-supervised classification with graph convolutional networks. arXiv preprint arXiv:1609.02907.
Kipf, T.N., Welling, M., 2016b. Variational graph auto-encoders. arXiv preprint arXiv:1611.07308.
Li, X., Cao, X., Qiu, X., Zhao, J., Zheng, J., 2017. Intelligent anti-money laundering solution based upon novel community detection in massive transaction networks on spark. In: 2017 Fifth International Conference on Advanced Cloud and Big Data (CBD). IEEE, pp. 176–181.
Liu, X., Zhang, P., Zeng, D., 2008. Sequence matching for suspicious activity detection in anti-money laundering. In: International Conference on Intelligence and Security Informatics. Springer, pp. 50–61.
Lo, W.W., Kulatilleke, G.K., Sarhan, M., Layeghy, S., Portmann, M., 2023. Inspection-l: self-supervised GNN node embeddings for money laundering detection in bitcoin. Applied Intelligence 1–12.
Lorenz, J., Silva, M.I., Aparício, D., Ascensão, J.T., Bizarro, P., 2020. Machine learning methods to detect money laundering in the bitcoin blockchain in the presence of label scarcity. In: Proceedings of the First ACM International Conference on AI in Finance, pp. 1–8.
Pareja, A., Domeniconi, G., Chen, J., Ma, T., Suzumura, T., Kanezashi, H., Kaler, T., Schardl, T., Leiserson, C., 2020. Evolvegcn: evolving graph convolutional networks for dynamic graphs. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 34, pp. 5363–5370.
Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Kopf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., Chintala, S., 2019. Pytorch: an imperative style, high-performance deep learning library. In: Wallach, H., Larochelle, H., Beygelzimer, A., d'Alché-Buc, F., Fox, E., Garnett, R. (Eds.), Advances in Neural Information Processing Systems, vol. 32. Curran Associates, Inc., pp. 8024–8035
Perozzi, B., Al-Rfou, R., Skiena, S., 2014. Deepwalk: online learning of social representations. In: Proceedings of the 20th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 701–710.
Pol, R.F., 2020. Anti-money laundering: the world's least effective policy experiment? Together, we can fix it. Policy Design and Practice 3, 73–94.
PwC, 2010. From Source to Surveillance: the Hidden Risk in AML Monitoring System Optimization. Technical Report PricewaterhouseCoopers LLP. URL: https://www.pwc.com/us/en/anti-money-laundering/publications/assets/aml-monitoring-system-risks.pdf.
Savage, D., Wang, Q., Zhang, X., Chou, P., Yu, X., 2017. Detection of money laundering groups: supervised learning on small networks. In: AAAI Workshops.
Schlichtkrull, M., Kipf, T.N., Bloem, P., Berg, R.v. d., Titov, I., Welling, M., 2018. Modeling relational data with graph convolutional networks. In: European Semantic Web Conference. Springer, pp. 593–607.
United Nations – Office on Drugs and Crime, 2011. Estimating Illicit Financial Flows Resulting from Drug Trafficking and Other Transnational Organized Crime.
Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., Bengio, Y., 2017. Graph attention networks. arXiv preprint arXiv:1710.10903.
Wang, C., Pan, S., Long, G., Zhu, X., Jiang, J., 2017. Mgae: marginalized graph autoencoder for graph clustering. In: Proceedings of the 2017 ACM on Conference on Information and Knowledge Management, pp. 889–898.
Wang, X., Ji, H., Shi, C., Wang, B., Ye, Y., Cui, P., Yu, P.S., 2019. Heterogeneous graph attention network. In: The World Wide Web Conference, pp. 2022–2032.
Weber, M., Chen, J., Suzumura, T., Pareja, A., Ma, T., Kanezashi, H., Kaler, T., Leiserson, C.E., Schardl, T.B., 2018. Scalable graph learning for anti-money laundering: a first look. arXiv preprint arXiv:1812.00076.
Weber, M., Domeniconi, G., Chen, J., Weidele, D.K.I., Bellei, C., Robinson, T., Leiserson, C.E., 2019. Anti-money laundering in bitcoin: experimenting with graph convolutional networks for financial forensics. arXiv preprint arXiv:1908.02591.
Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Philip, S.Y., 2020. A comprehensive survey on graph neural networks. IEEE Transactions on Neural Networks and Learning Systems 32, 4–24.
Zhang, C., Song, D., Huang, C., Swami, A., Chawla, N., 2019. Heterogeneous graph neural network. In: Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining.
Yang, C., Xiao, Y., Zhang, Y., Sun, Y., Han, J., 2020. Heterogeneous network representation learning: a unified framework with survey and benchmark. IEEE Transactions on Knowledge and Data Engineering 34 (10), 4854–4873.
Zhang, M., Chen, Y., 2018. Link prediction based on graph neural networks. Advances in Neural Information Processing Systems 31.
Zhang, Y., Trubey, P., 2019. Machine learning and sampling scheme: an empirical study of money laundering detection. Computational Economics 54, 1043–1063.